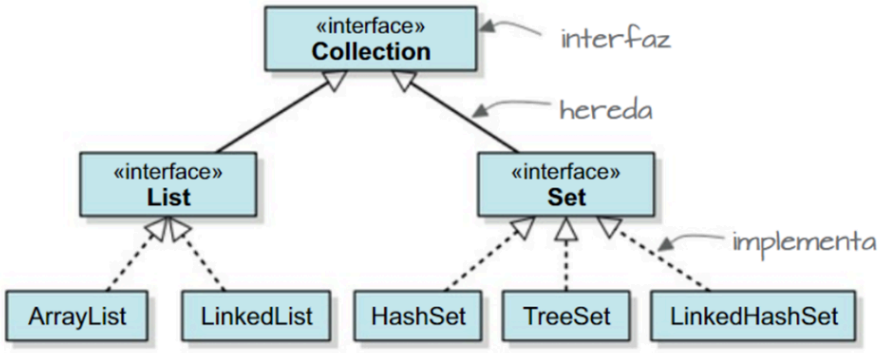
Unidad 11 Colecciones

Interfaz Collection
Listas
Métodos de la interfaz Collection
Métodos básicos de la interfaz Collection
Métodos globales de la interfaz Collection
Métodos de tabla de la interfaz Collection

Java nos proporciona una serie de estructuras dinámicas que comparten un conjunto de métodos declarados en la interfaz Collection. Todas ellas implementan dicha interface, aunque de distinta forma.



Una colección es un objeto que sirve para agrupar un conjunto de objetos, llamados elementos, que generalmente guardan una relación entre ellos.

Hay colecciones de diferentes tipos, adaptadas a distintos fines más o menos específicos. Por eso no existe una clase colección, sino todo un marco de trabajo (framework)

Los tipos fundamentales de estructuras dentro del framework de las colecciones son tres:

- **Listas:** responden a la necesidad de manejar sucesiones de datos que pueden estar repetidos y cuyo orden puede ser relevante. De alguna forma sustituyen a las tablas, con la diferencia de que podemos insertar o eliminar datos en ellas sin limitaciones de espacio. Implementan la interfaz List que, al heredar de Collection, incorpora todas sus funcionalidades y añade alguna más.

- **Conjuntos**: el orden de los datos no es relevante y lo que realmente importa es la mera pertenencia a la estructura, con lo que las repeticiones

ni son posibles ni tienen sentido. Implementan la interfaz Set, que también hereda de Collection.

- **Mapas o diccionarios**: están dentro del framework de las colecciones, aunque no implementan la interfaz Collection. Sirven para guardar datos identificados por claves que no se repiten. Los mapas implementan la interfaz Map, que no hereda de Collection.

De todas estas estructuras, la más conocida y usada es la lista, aunque como vemos, no es la única.

Todas ellas son dinámicas, ya que permiten añadir y quitar unidades de información (objetos llamados nodos o elementos) en tiempo de ejecución, sin más límite que la
memoria del ordenador. Nosotros llamaremos colecciones a todas las clases que implementen la interfaz Collection, aunque también usaremos el nombre del tipo especial de colección: lista o conjunto.

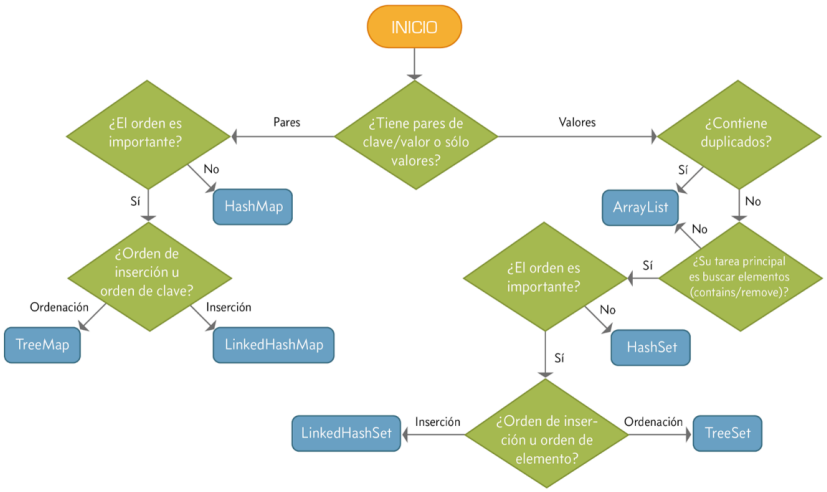
Hablamos del **framework Collection** como el entorno de trabajo formado por un conjunto de interfaces y clases relacionadas y que interactúan entre sí, pudiéndose obtener unas de otras según el modelo de datos que buscamos.

Interfaz Collection

La interfaz Collection define las funcionalidades comunes a todas las colecciones, ya sean listas o conjuntos. Sin embargo, las clases de la API que la implementan son listas o conjuntos, es decir, implementan la interfaz List o Set, que son extensiones de Collection. Ninguna implementa Collection directamente. Por ello, para poner ejem plos prácticos de ella, tendremos que usar listas o conjuntos.

Llamaremos colección a toda instancia de alguna de las clases que implementan la interfaz Collection. Estas incluyen: ArrayList, LinkedList, HashSet, TreeSet y LinkedHashSet. Los mapas (HashMap, TreeMap y LinkedHashMap) no son colecciones, aunque guardan relación con ellas.

Diagrama de decisión de uso



Listas

Las listas son clases que implementan la interfaz List, y sirven para almacenar datos que se pueden repetir y cuyo orden de inserción puede ser relevante.
Hay dos implementaciones de List, las clases ArrayList y LinkedList. Las dos proporcionan los mismos métodos y funcionalidades. La diferencia entre ArrayList y LinkedList radica en la implementación interna y solo afecta levemente al rendimiento. La primera es más rápida en las operaciones que supongan recorrer la lista para la lectura o modificación de elementos, mientras que la segunda tiene mejor rendimiento en las operaciones de inserción y eliminación de elementos.

La sintaxis para construir un objeto ArrayList con un tipo genérico de datos E (Cliente, Empleado, Integer...) es:

```
ArrayList<E> lista = new ArrayList<>();
```

De forma más general, dado que tanto la clase ArrayList como LinkedList implementan todos los métodos de la interfaz List, es posible utilizar una variable de este tipo para referenciar objetos de ambas clases.

```
List<E> lista = new ArrayList<>();
```

En esta lista solo se podrán insertar objetos (elementos) del tipo E.

Ejemplo

construcción de un ArrayList que nos sirva para guardar objetos de tipo Cliente

```
List<Cliente> listaClientes = new ArrayList<>();
```

Una vez creada la colección listaClientes, disponemos de una estructura dinámica donde insertar o eliminar objetos Cliente. Antes vamos a definir una clase Cliente que nos permita probar los distintos métodos con ejemplos concretos

La clase Cliente sería:

```
class Cliente implements Comparable<Cliente> {
    String dni;
    String nombre;
    LocalDate fechaNacimiento;
    Cliente(String dni, String nombre, String fechaNacimiento) {
        this.dni = dni;
        this.nombre = nombre;
        DateTimeFormatter formatoFechas=
                DateTimeFormatter.ofPattern("dd/MM/yyyy");
        this.fechaNacimiento = LocalDate.parse(fechaNacimiento, formatoFechas);
    }
    int edad(){
           return (int)fechaNacimiento.until(LocalDate.now(), ChronoUnit.YEARS);
    }
    @Override
    public boolean equals(Object ob) {
        return dni.equals(((Cliente) ob).dni);
    }
    @Override
    public int compareTo(Cliente otro) {
        return dni.compareTo(otro.dni);
    }
    @Override
    public String toString() {
        return "DNI: " + dni + "  Nombre: " + nombre + "  Edad: "  + edad() + "\n";
    }
}
```

Para estudiar la interfaz Collection, dado que la interfaz List hereda de ella, vamos a referenciar la lista **listaClientes**

```
List<Cliente> listaClientes = new ArrayList<>();
```

con la variable **coleccionClie** del tipo Collection. Con ello conseguiremos tener acceso únicamente a las funcionalidades de Collection, que son las que vamos a estudiar a continuación, y no a las específicas de la interfaz List que veremos luego.

```
Collection<Cliente> coleccionClie = listaClientes;
```

Mientras usemos la variable **coleccionClie**, el objeto referenciado (una lista) se comportará como una simple colección, sin ninguna de las particularidades específicas de las listas

Métodos de la interfaz Collection

Los métodos de la interfaz Collection son de dos tipos. Unos afectan a elementos individuales y otros a grupos de elementos. A los primeros los llamaremos métodos básicos y a los segundos, métodos globales

Haremos las pruebas sobre la lista creada anteriormente.

```
class Cliente implements Comparable<Cliente> {
    String dni;
    String nombre;
    LocalDate fechaNacimiento;
    Cliente(String dni, String nombre, String fechaNacimiento) {
        this.dni = dni;
        this.nombre = nombre;
        DateTimeFormatter formatoFechas=
                DateTimeFormatter.ofPattern("dd/MM/yyyy");
        this.fechaNacimiento = LocalDate.parse(fechaNacimiento, formatoFechas);
    }
    int edad(){
           return (int)fechaNacimiento.until(LocalDate.now(), ChronoUnit.YEARS);
    }
    @Override
    public boolean equals(Object ob) {
        return dni.equals(((Cliente) ob).dni);
    }
    @Override
    public int compareTo(Cliente otro) {
        return dni.compareTo(otro.dni);
    }
    @Override
    public String toString() {
        return "DNI: " + dni + "  Nombre: " + nombre + "  Edad: "  + edad() + "\n";
    }
}
```

```
List<Cliente> listaClientes = new ArrayList<>();
Collection<Cliente> coleccionClie = listaClientes;
```

Métodos básicos de la interfaz Collection

Métodos de inserción

boolean add(E elem): Añade un nuevo elemento a la lista

```
Cliente cliente = new Cliente("111", "Marta", "12/02/2000" );
coleccionClie.add(cliente);
```

Métodos de eliminación

boolean remove(Object ob): Elimina la primera ocurrencia del elemento indicado. Se puede observar que no se exige a ob que sea del tipo genérico E con el que se ha declarado la colección. Esto se debe a que el método no va a añadir ningún elemento, y no hay peligro de que se inserte un objeto de una clase no permitida.

```
coleccionClie.remove(cliente);
```

void clear(): elimina todos los elementos de una colección y la deja vacía. Esto no significa eliminar la propia colección. La colección, simplemente, queda vacía y disponible para volver a insertar nuevos elementos.

```
coleccionClie.clear();
```

Métodos de comprobación

int size(): nos permite saber, en cada momento, el número de elementos

```
coleccionClie.add(new Cliente("111", "Marta", "12/02/2000"));
coleccionClie.add(new Cliente("115", "Jorge", "16/03/1999"));
coleccionClie.add(new Cliente("112", "Carlos", "01/10/2002"));

coleccionClie.size(); //devuelve 3
```

boolean isEmpty(): permite saber si una colección está vacía.

```
coleccionClie.isEmpty(); //devolverá false
```

boolean contains(Object ob): nos dice si un elemento ob determinado está en una colección. Devuelve true si ob pertenece a la colección y false en caso contrario.

```
coleccionClie.contains(new Cliente("115", "Jorge", "16/03/1999")); //true
```

En la búsqueda del objeto ob (llamado clave de búsqueda), contains() usa el método equals() para compararlo con los elementos de la colección. En nuestro ejemplo con objetos Cliente, ese método está basado en el atributo dni. Por tanto, la expresión

```
coleccionClie.contains(new Cliente("115", "", ""));
```

devolverá true, ya que, en la búsqueda del objeto, el programa solo va a comparar el atributo dni de la clave de búsqueda con los de los distintos elementos de la colección.

Otros métodos

String toString(): devuelve una cadena que representa la colección. Todas las colecciones tienen implementado este método, que muestra los elementos entre corchetes y separados por comas. Cada elemento se muestra, a su vez, según la implementación de toString() que tenga la clase E.

```
System.out.println(coleccionClie);
```

```
[DNI: 111 Nombre: Marta Edad: 20
, DNI: 115 Nombre: Jorge Edad: 21
, DNI: 112 Nombre: Carlos Edad: 18
]
```

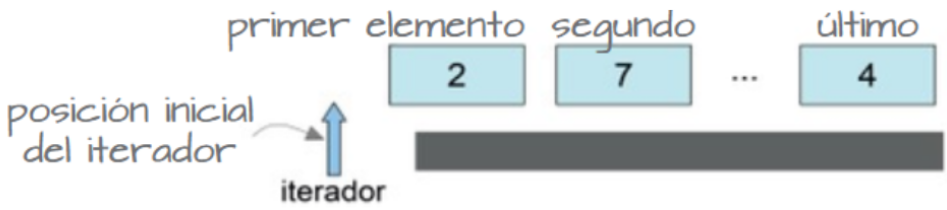
Formas para recorrer la colección

A menudo necesitamos recorrer una colección elemento a elemento. Una de las formas de hacerlo es por medio de iteradores, que son objetos que van apuntando sucesivamente a los elementos de la colección, empezando por el primero. Para usar un iterador con una colección concreta, primero hay que crearlo.

Iterator<E> iterator(): invocado por la colección, nos devuelve un iterador asociado a ella. Aquí E será del mismo tipo que el de la colección.

```
Iterator<Cliente> it = coleccionClie.iterator();
```

it es el iterador que sirve para recorrer **coleccionClie**. Para ello dispone de los métodos hasNext() y next() , que se complementan y emplean conjuntamente. Inicialmente it apunta al principio de la colección, justo antes del primer elemento.



boolean hasNext(): comprueba si hay un elemento siguiente, es decir, si quedan elementos por visitar, y nos devuelve true o false, según el caso.

E next(): el iterador avanza al siguiente elemento, si existe, y nos lo devuelve. En caso de que no haya siguiente, porque estemos al final de la colección o porque esta estuviera vacía, next() lanzará la excepción NoSuchElement Exception. La primera llamada a next() nos devuelve el primer elemento de la colección si esta no está vacía.

En la práctica, para evitar la excepción, los dos métodos se usan conjuntamente, de forma que solo se llama al método next() si antes se ha comprobado, con hasNext(), que hay elemento siguiente

```
ArrayList<String> frutas = new ArrayList<>();
frutas.add("Fresa");
frutas.add("Pera");
frutas.add("Kiwi");
frutas.add("Melón");

Iterator<String> it = frutas.iterator();

while(it.hasNext()) {
    String fruta = it.next();
    System.out.println(fruta);
}
```

void remove(): elimina de la colección el último elemento devuelto por next(), que es el apuntado por el iterador en cada momento. Así podemos eliminar un elemento de una colección dependiendo de alguna condición mientras se está recorriendo con un iterador. Esta es la única forma de eliminar un elemento preservando la integridad de la colección. Aunque tenga el mismo nombre, no debemos confundirlo con el método remove (Object ob) de la interfaz Collection, que se invoca desde un objeto que implementa colección y no debe usarse dentro

del bucle de un iterador. En este contexto, debemos invocar el del iterador, al que no se le pasa ningún parámetro.

```
it = frutas.iterator();  // Reiniciar
while(it.hasNext()) {
    String fruta = it.next();
    if (fruta.length()==5)
         it.remove();
}
```

```
it = frutas.iterator(); // Reiniciar
while(it.hasNext()) {
    String fruta = it.next();
    System.out.println(fruta);
}
```

Con el caso clientes, si queremos eliminar de nuestra colección aquellos clientes nacidos antes del año 2000.

```
Iterator<Cliente> it = coleccionClie.iterator();
while(it.hasNext()) {
          Cliente p = it.next();
          if (p.fechaNacimiento.compareTo(LocalDate.of(2000, 1, 1))< 0){
                  it.remove(); /*elimina p, ultimo cliente devuelto por next()
                  ¡No usar coleccionClie.remove(p)!*/
          }
}
```

Una forma mucho más simple de recorrer una colección es por medio de la estructura for extendido o for-each

```
for (Cliente c : coleccionClie) {
        System.out.println(c);
}
```

Para cada cliente c que pertenece a **coleccionClie**, mostrar c. En este bucle, la variable local c, de tipo Cliente, va tomando todos los valores de la colección. Sin embargo, hay operaciones para las que un bucle for extendido no vale. Por ejemplo, no podemos eliminar elementos, ya que c es siempre la referencia a una copia de un elemento de la colección. Para la eliminación de elementos durante el recorrido de una colección, tenemos que usar un iterador.

Métodos globales de la interfaz Collection

Los métodos básicos afectan a un solo elemento. Existen otros métodos, llamados métodos globales, en los que intervienen más elementos, incluso más de una colección.

boolean containsAll(Collection<?> c): se le pasa como parámetro otra colección que es de un tipo genérico cualquiera, independientemente del tipo E de la colección que hace la llamada. Devuelve true si todos los elementos de c están en la colección que hace la llamada y false si hay al menos un elemento de c que no está en ella.

Primera colección

```
List<Cliente> listaClientes = new ArrayList<>();
Collection<Cliente> coleccionClie = listaClientes;
coleccionClie.add(new Cliente("111", "Marta", "12/02/2000"));
coleccionClie.add(new Cliente("115", "Jorge", "16/03/1999"));
coleccionClie.add(new Cliente("112", "Carlos", "01/10/2002"));
```

Segunda colección

```
Collection<Cliente> otrosClientes = new ArrayList<>();
otrosClientes.add(new Cliente("111", "Marta", "12/02/2000"));
otrosClientes.add(new Cliente("112", "Carlos", "01/10/2002"));
coleccionClie.containsAll(otrosClientes);
```

Esto devuelve true porque todos los del segundo ya están en el primero, pero si añadimos un elemento al segundo que no esté en el primero devolverá false.

boolean addAll(Collection<? extends E> c): añade a la colección que hace la llamada todos los elementos de la colección c. La forma en que se añaden depende de la implementación concreta de la colección. Si es una lista, se añadirán todos al final, aunque estén repetidos. Si la implementación es un conjunto, no se añaden los repetidos ni tienen por qué insertarse al final. El tipo de c es E o una subclase.

```
coleccionClie.addAll(otrosClientes);
```

Si los mostramos salen todos los elementos de ambas colecciones

boolean removeAll(Collection<?> c): elimina de la colección invocante todos los elementos que estén contenidos en c. Después de ejecutar el método no habrá elementos comunes a las dos colecciones.

boolean retainAll(Collection<?> c): elimina todos los elementos de la colección invocante, salvo aquellos que también estén en c.

Métodos de tabla de la interfaz Collection

Hay una tercera categoría de métodos que comparten todas las colecciones: aquellos que sirven para volcar sus datos en tablas.

Object[ ] toArray(): devuelve una tabla de tipo Object con los mismos elementos de la colección. En el caso de listas, como en estas el orden importa, la tabla alberga los mismos elementos, incluyendo las repeticiones, en el mismo orden.

```
Object[] arrayClientes = listClientes.toArray();
((Cliente) arrayClientes[0]).nombre;
```

<T>T[ ] toArray (T[ ] t): es igual que el anterior, pero devuelve una tabla de tipo genérico T. El método es invocado por la colección de tipo genérico T y, como parámetro, le pasamos una tabla del mismo tipo. No hay que inicializar la tabla ni importa su tamaño. El método la devuelve redimensionada con el tamaño necesario para albergar todos los elementos de la colección. De hecho, es costumbre definirla con tamaño O.

```
Cliente[] arrayClientes = listClientes.toArray(new Cliente[0]);
arrayClientes[0].nombre;
```